SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0305	**Course Title:**	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	**Assessment:**	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	**Total Marks:**	100
CO2 Statement:	*Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.*		
Student Name:	Siva Charan	**Reg. No.:**	192525039
Section / Batch:	2025	**Date:**	20/07/2026

Code of Conduct

I Siva Charan (192525039) *(Name / Reg No)* *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Siva Charan

Instructions

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
		GRAND TOTAL:	**100 Marks**

			function to print the same in reverse order using stack data structure.
21	192525063	NALIKIRI SIVA VENKATA TRIVIKRAM	1. Write an algorithm called Find-Largest that finds the largest number in an array using a divide-and-conquer strategy. 2. Analyze how to print all the prime integers present in the Single linked list having a list of 24->21->57->31->12->47->7->38->.
22	192511294	NIRUTHIKA SRI PRAKASH	1. Perform the following operations in LIFO data structure having an array size of 5. Push(45), Push(22), Push(11), Push(77), Pop(), Push(44), Push(33), Pop(), Pop(),Pop(). Print the peak element of the data structure and its index position. With a neat sketch explained in detail. 2. How do you implement Queue data structure using Single Linked List? With a neat diagram discussing Enqueue and Dequeue operations using SLL.
23	192525039	OSURU SIVA CHARAN	1. Perform the following operations in Circular Queue having a size of 5. Enqueue(20), Enqueue(10), Enqueue(30), Dequeue(), Enqueue(80), Dequeue(), Enqueue(90), Enqueue(100), Dequeue(), Dequeue(), Enqueue(60), Enqueue(90). Finally display the front and rear elements with its positions. Discuss in detail. 2. Implement Binary searching Algorithm for searching an element and its position having the following 10 elements. Elements are 66, 111, 212, 323, 432, 543, 643, 649, 777, 888. Case 1: Key element = 111, Case 2: Key element=888, Case 3: Key element = 999. Analyse the concept in detail.
24	192572014	PRITHAM G	1. Pictorially insert an element at 3rd position in a doubly linked list, having 6 elements in the list and delete the element at 2nd position from the list. Implement and analyse it. 2. Perform the following operations using stack. Assume the size of the stack is 5 and have a value of 22, 55, 33, 66, 88 in the stack from 0 position to size-1. Now perform the following operations: 1) Invert the elements in the stack, 2) Pop(), 3) Pop(), 3) Pop(), 4) Push(90), 5) Push(36), 6) Push(11), 7) Push(88), 8) Pop(), 9) Pop(). Draw the diagram of the stack and illustrate the above operations and identify where the top is?
25	192525195	R BHAVYA	1. Convert the following infix expression into a post fix expression. B - (C/D + (E % F* G) / H) * J. Also, evaluate the given postfix expressions. a) 9 3 8 4 /*+ b) 6 2 + 6 3 2. Illustrate the queue operation using following function calls of size = 5. Enqueue(21), Enqueue(67), Enqueue(89), Dequeue(), Enqueue(15), Enqueue(30), Enqueue(52), Dequeue(), Dequeue(), Dequeue(),Dequeue().

38	192524251	VAISHNAV M	1. Two algorithms solve the same problem. Algorithm A uses recursion, while Algorithm B uses an explicit stack. Compare both approaches in terms of memory usage, execution flow, and risk of stack overflow. 2. A stack has a capacity of 5 elements. Explain what happens if the following operations are executed: Push(A), Push(B), Push(C), Push(D), Push(E), Push(F). Identify the error and suggest two possible solutions.
39	192525187	VEERA SANTHOSH A	1. Compare the performance of a linear queue and a circular queue when 100 enqueue and dequeue operations are performed alternately. 2. A queue implemented using an array report "Queue Full" even though some positions are empty. ➢ Analyze why this occurs. ➢ Explain how a circular queue solves this problem.
40	192511226	VINIL KUMAR S	1. Consider the singly linked list: 10 → 20 → 30 → 40 → 50 Delete the node containing 30. Explain how the links change after deletion. 2. Given two sorted linked lists: L1: 1 → 3 → 5 → 7 L2: 2 → 4 → 6 → 8

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature:	Signature: ________	Signature: ________
Date: ________	Date: ________	Date: ________

CO2 - AT1 - Analytical Problem

Siva Charan
192525039

Circular Queue (Size = 5)

Initial condition

Queue size = 5

Position	0	1	2	3	4
Elements	-	-	-	-	-

front = -1

Rear = -1

Operation 1: Enqueue (20)

Queue is Empty, so both front & rear become 0.

0	1	2	3	4
20	-	-	-	-

Operation 2: Enqueue (10)

Rear = (0+1) % 5 = 1

0	1	2	3	4
20	10	-	-	-

Operation 3: Enqueue (30)

Rear = (1+1) % 5 = 2

0	1	2	3	4
20	10	30	-	-

Operation 4: Dequeue ()

Element removed = 20

front = (0+1) % 5 = 1

0	1	2	3	4
-	10	30	-	-

Operation 5: Enqueue (80)

Rear : (2+1) % 5 = 3

0	1	2	3	4
-	10	30	80	-

operation 6 : Dequeue ()

Element removed = 10

Front = (1+1) % 5 = 2

0	1	2	3	4
-	-	30	80	-

operation 7 : Enqueue(90)

Rear : (3+1) % 5 = 4

0	1	2	3	4
-	-	30	80	90

operation 8 : Enqueue (100)

Rear = (4+1) % 5 = 0

0	1	2	3	4
100	-	30	80	90

operation 9 : Dequeue ()

Element removed = 30

0	1	2	3	4
100	-	-	80	90

operation 10 : Dequeue ()

Element removed = 80

Front = (3+1) % 5 = 4

0	1	2	3	4
100	-	-	-	90

operation 11 : Enqueue (60)

Rear = (0+1) % 5 = 1

0	1	2	3	4
100	60	-	-	90

operation 12 : Enqueue(90)

Rear = (1+1) % 5 = 2

0	1	2	3	4
100	60	90	-	90

Final Queue 90 -> 100 -> 60 -> 90

Position	0	1	2	3	4
Element	100	60	90	-	90

...ary search Algorithm is an efficient searching technique ...ed to find an element in a sorted array. It compares the ...key with the middle element and eliminates half of the elements in each step.

Given Elements

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

Algorithm

1) Set low = 0 and high = n-1.
2) Calculate mid = (low + high) / 2.
3) Compare the key with A[mid].
4) If equal, the element is found.
5) If key is smaller, search the left half.
6) If key is larger, search the right half.

Case 1 : key = 111

- Middle = 432
- 111 < 432 -> search left half.
- Middle of left half = 111
- Found at Position 2 (index 1).

Case 2 : key = 888

- Middle = 432 -> search right.
- Middle = 649 -> search right.
- Middle = 777 -> search right.
- Middle = 888 -> found.
- Found at Position 10 (index 9).

: Key = 999

- Middle = 432 → search right.
- Middle = 649 → search right.
- Middle = 777 → search right.
- Middle = 888 → 999 is greater.
- No elements remain.
- 999 is not found; Position = -1.

Final result

Key	Result	Position
111	found	2
888	found	10
999	not found	-1

Analysis

Binary search requires a sorted array. Its best-case complexity is O(1), while average and worst-case complexity is O(log n).